

This approach is simple because:

- No coding is involved.
- Performance hit is minimal.
- It can be used against any function in the application, provided the function side-effects do not alter the overall program flow.
- No debug information is required in binary; it works on release-mode binaries, as long as they are unstripped.

## Automating the attack: Gdb scripts at work

*Gdb* can be invoked in batch mode, wherein it reads inputs from a file (called a *gdb* script) instead of reading interactively. We will be looking into two *gdb* options, *-batch* and *-x*. The *-x* switch takes the name of a file from where *gdb* reads commands. Please refer to the *gdb* man page for more details.

Let us write a script with *gdb* commands in it, to automate the steps we performed in the previous section. We can name our script *hackScript.gdb*, and here are its contents:

```
file ./hackMe
br authenticate
r FAIL
printf "\n ----> Breakpoint HIT...\n"
stepi
step
printf "\n\n ----> I AM OUT OF THE TARGET FUNCTION, BACK IN MAIN: RETURN VALUE HERE(BEFORE HACK) IS %d", $eax
printf "\n\n ----> SETTING THIS VALUE TO 0\n"
set $eax=0
printf "\n\n ----> EAX AFTER HACK IS %d", $eax
c
```

This script loads the binary, and then inserts a breakpoint with the command *br*. Let us use the *gdb* command *printf* (similar to *libc printf*) to print logging information. Given below is the output to run this script in *gdb*:

```
[raman@Linux32 article]$ gdb -x hackScript.gdb -batch
Breakpoint 1 at 0x0048396

Breakpoint 1, 0x0048396 in authenticate ()

----> Breakpoint HIT...
0x0048399 in authenticate ()
Single stepping until exit from function authenticate,
which has no line number information.
0x004840e in main ()

----> I AM OUT OF THE TARGET FUNCTION, BACK IN MAIN: RETURN
VALUE HERE(BEFORE HACK) IS 1

----> SETTING THIS VALUE TO 0
```

```
----> EAX AFTER HACK IS 0
Authenticated ...program continuing...
```

## Counter measures

1. Obfuscation—this technique involves changing the function names to some hard-to-guess names. This can be achieved by just using the *#define* construct in C.
2. Use function side-effects—don't just make decisions on the return value, use an additional criterion.
3. Use a challenge-response mechanism.
4. Strip the binary so that it's difficult to determine symbols (functions, variables, etc) and their address in the binary.
5. Include fake code before exiting the application, so that it's hard to detect the location where the program exits.

We learned how *gdb* can be used to hack an application if the hacker has sufficient information about the key functions. Although key functions can be obfuscated, some level of R&D with the application can undermine the security of the application. The application used in this article was built in release mode, though unstripped. The vulnerability can be reduced by stripping (with the *strip* command) before shipping. However, even stripping the binary may not be very effective, as some smart minds have an *UNSTRIP* program. Stripping a binary may also result in debugging challenges in the field.

## What next

I hope you find this article useful for educational purposes. The idea discussed can be extended (in a positive manner) to build a logging mechanism for a particular application, in order to debug the behaviour of the application—and, in some cases, alter it! In the next article, we will try to make better use of this technique by digging a little deeper into the *glibc* interface layer, and explore more threats that our application will have to face in the real world! 

## Disclaimer

The information provided in this article is only for educational purposes and should not be used to attack/hack any application that is not owned by you.

## References

1. *gdb* man page.
2. *unstrip*: <http://www.paradyn.org/html/tools/unstrip.html>

## By: Raman Deep

Raman works as a senior software engineer in a company that deals in software security. He has six years of software development experience in C and Linux. He can be reached at [rd.golinux@gmail.com](mailto:rd.golinux@gmail.com).